

MixT: A Language for Mixing Consistency in Geodistributed Transactions

Mae Milano

Dept. of Computer Science, Cornell University
Ithaca, New York, United States of America
milano@cs.cornell.edu

Andrew C. Myers

Dept. of Computer Science, Cornell University
Ithaca, New York, United States of America
andru@cs.cornell.edu

Abstract

Programming concurrent, distributed systems is hard—especially when these systems mutate shared, persistent state replicated at geographic scale. To enable high availability and scalability, a new class of weakly consistent data stores has become popular. However, some data needs strong consistency. To manipulate both weakly and strongly consistent data in a single transaction, we introduce a new abstraction: mixed-consistency transactions, embodied in a new embedded language, MixT. Programmers explicitly associate consistency models with remote storage sites; each atomic, isolated transaction can access a mixture of data with different consistency models. Compile-time information-flow checking, applied to consistency models, ensures that these models are mixed safely and enables the compiler to automatically partition transactions. New run-time mechanisms ensure that consistency models can also be mixed safely, even when the data used by a transaction resides on separate, mutually unaware stores. Performance measurements show that despite their stronger guarantees, mixed-consistency transactions retain much of the speed of weak consistency, significantly outperforming traditional serializable transactions.

CCS Concepts • Information systems → Distributed database transactions; • Software and its engineering → Consistency; Domain specific languages;

Keywords Consistency, Transactions, Information Flow

ACM Reference Format:

Mae Milano and Andrew C. Myers. 2018. MixT: A Language for Mixing Consistency in Geodistributed Transactions. In *Proceedings of 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI'18)*. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3192366.3192375>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. PLDI'18, June 18–22, 2018, Philadelphia, PA, USA

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-5698-5/18/06.

<https://doi.org/10.1145/3192366.3192375>

1 Introduction

Programmers often face the task of writing concurrent, distributed systems that share mutable, persistent state across geographic distances. Traditional tools, such as strictly serializable atomic database transactions and distributed locking, do not scale across continents; the speed of light simply isn't fast enough for the cross-continental round trips needed by traditional transactions.

New tools have evolved to fill the gap, relying on weaker consistency to enable lower latencies and higher availability. Evidence from both industry [8, 20, 31, 35, 46] and academia [13, 27, 51] suggests that weak consistency is viable, especially when accompanied by familiar transactional abstractions, clear consistency guarantees, and efficient operations over persistent data.

But problems remain. While weaker consistency models suffice for certain application data, other data needs stronger consistency—single applications might need multiple *levels* of consistency. Running with a single consistency level, as seen in prior systems [7, 15, 29, 56, 58, 59]), can be slow [5]; all operations within a transaction must be upgraded to the consistency required by the most sensitive among them, introducing unnecessary delay and contention for objects that only require weak consistency. This is a fundamental problem: we need a general way to construct transactions that access data at multiple consistency levels, without compromising strong consistency where it is needed.

These concerns lead us to a key observation: consistency is a property of information itself and not only of operations that use this information. Further, the consistency with which we manipulate information should always match or exceed the consistency at which we store it. Based on this observation, we introduce *mixed-consistency transactions*. Each mixed-consistency transaction can operate over any and all data, even if this data is stored with varying consistency guarantees. Despite this expressivity, we maintain the consistency guarantees required by each data object by preventing less-consistent information from influencing more-consistent data. Mixed-consistency transactions are atomic: no operations inside a transaction become visible outside the transaction until all operations do.

In implementing mixed-consistency transactions, we uncover a further complication: engineers at major companies frequently find themselves writing distributed programs that

mix accesses to data from multiple existing storage systems, each with distinct consistency guarantees [6]. It is unrealistic to assume that data can be freely migrated into ever-newer and more capable storage systems, or that all applications can be written against a single unified system; we therefore want to operate against multiple backing stores within the same mixed-consistency transaction.

Combining these challenges, we present MixT, a domain-specific language for mixed-consistency transactions. In MixT, persistent data and operations at various stores can be accessed with strong guarantees (§3). To ensure the semantic guarantees of mixed-consistency transactions, weaker-consistency information should avoid influencing stronger-consistency information. To prevent this influence, we view consistency as a property of data, treating consistency as a form of data integrity [4] expressed as labels on types in the language. Static analysis of information flow [47] then ensures that consistency guarantees cannot be violated by exposure to objects with weaker consistency.

The MixT language implements mixed-consistency transactions using three novel mechanisms (§4–5):

- Compile-time information flow control ensures that the consistency of data is never weaker than the level described by its storage location.
- Using information flow analysis, the code of each transaction is automatically *split* into sub-transactions for each consistency level, while preserving atomicity.
- A lightweight run-time mechanism ensures transactional atomicity, even between sub-transactions executing on multiple mutually unaware backing stores.

MixT works on top of stores' existing transactional mechanisms, without changing the representation of existing data, allowing existing applications to operate unmodified alongside MixT applications. And MixT can be easily adapted to a new store, by inserting the store's consistency level into MixT's consistency lattice and providing bindings to custom data operations specific to that store.

As we show experimentally (§8), mixed-consistency transactions perform well. MixT enables significant speedup vs. serializable transactions by exploiting weak consistency, without losing the guarantees sacrificed by current systems when consistency levels mix.

2 Motivation

2.1 A Running Example

Suppose we are building a scalable group messaging service called *Message Groups*. This service allows users to join groups and to post messages to all members of any group they have joined. For low-latency communication, application servers are deployed across the world, with data replicated across these servers.

Communication latency between these servers makes it difficult to keep the replicated data fully consistent without degrading user experience. Fortunately, there is no need to enforce a global, total order on displayed messages. It is only necessary to respect potential causality, so that messages precede their responses. We therefore geo-replicate user inboxes at a weaker consistency level, *causal+ consistency*, which respects causality but does not guarantee a total order [39].

However, other data in this application requires stronger consistency. The membership of users in various groups should be consistent worldwide so that all servers agree on who is supposed to receive which messages. Therefore, the set of members of each group is placed at a store supporting *linearizable transactions*, which ensure serializability and external consistency [12, 25, 45]. Latency to this single store is necessarily much higher for many users than latency to their own inboxes.

2.2 The Need For Mixed-Consistency Transactions

To see the pitfalls inherent to this naive mixing of consistency levels, consider how Message Groups might implement logged message delivery, using the code in Figure 1. There is a linearizable list of members named *users* to whom a message *post* should be sent. Each member in *users* has a causally consistent inbox. For maintenance purposes, the sending of messages is logged (via *log.append*). The log does not even require causal consistency; instead, we might replicate it at *eventual consistency* [55], which requires only that reads converge after a sufficiently long quiescence. All mutations, including append and insert, are replicated across continents.

However, the simple loop in Figure 1 does not address concurrent modification to the data. Suppose that a thread concurrently modifies *users* while another thread is executing Figure 1. Without care, this concurrent modification might invalidate Figure 1's iterator; at best, it is unclear whether the new member will receive a message. As written, there is no reason to expect the result of this execution to be atomic, isolated, or even complete.

Clearly, some form of concurrency control is needed. We might change over to a recent system such as Quelea [51] or Salt [56], which provide both fully atomic transactions and multiple consistency levels. But these systems can only execute a given transaction at a single consistency level. In these systems, all data in the Message Groups example would effectively be upgraded to linearizability. There would be no performance benefit from having a weakly consistent inbox and log; message delivery performance would likely be unacceptable.

Alternatively, we could partition our data onto three distinct systems, each optimized for the appropriate consistency

```

var iterator = users,
while (iterator.isValid()) {
  log.append(iterator->v.inbox.insert(post)),
  iterator = iterator->next
}

```

Figure 1. Naive code for sending messages. Corrected MixT code is found in Figure 4.

```

if (a.inbox.size() >= 1000000 &&
    b.inbox.size() < 1000000) { // weak condition
  a.declare_winner()           // strong effect
} else
if (a.inbox.size() < 1000000 &&
    b.inbox.size() >= 1000000) {
  b.declare_winner()
}

```

Figure 2. Contest logic inside the mail delivery transaction. Corrected MixT code is found in Figure 13.

level. If we had a causal store such as TaRDiS [13] that supports interactive atomic transactions, we could start a separate simultaneous transaction at each system. This would achieve the desired performance, but the implicit interactions between these transactions could create bugs. For example, if another process updated the membership list while mail was being sent, the linearizable transaction might abort and roll back, restarting the loop. Without code to explicitly roll back the other concurrent transactions—which may not even always be possible—some users could then receive the same message a second time.

Thus, this transaction cannot be naturally implemented using each underlying store’s mechanisms in isolation. It requires a new form of *mixed-consistency*, *mixed-location* transaction not supported by any existing system, with new run-time mechanisms for atomicity across different consistency levels.

2.3 Mixing Consistency Breaks Strong Consistency

There is a reason that existing systems choose a single consistency level for each transaction. Causal consistency and linearizability offer well-defined consistency guarantees, but trying to mix these levels in the same application can break the guarantees that both levels claim to offer, even if all the issues in the previous section were solved. Some transactions simply are not safe to run under mixed consistency. To see why, consider the following example.

Suppose we run a contest to advertise Message Groups. Users are divided into two teams; team A sends messages to mailbox *a*, and team B sends messages to mailbox *b*. The first team to send 1,000,000 messages is declared the winner.

To implement this contest, we extend the existing transaction for delivering mail with a few lines of code shown in Figure 2. After running the contest, we may be surprised to discover that the code has not declared a unique winner; both teams A and B are simultaneously declared the winner!

This code has a fundamental problem. To avoid slowing down the core functionality of message delivery, the guard condition uses data (the inbox sizes) stored with only causal consistency. Since the guard is evaluated with causal consistency, nothing guarantees that the function `declare_winner()` is invoked only once. But the function `declare_winner()` manipulates only data with linearizable consistency; it is not designed to deal with the potential for multiple re-executions. During a partial network partition, every single message receipt to either team could cause the winner to switch, as the causal replica receiving messages for *a* may not be able to propagate events to the replica receiving messages for *b* (and vice-versa). This causes each team to believe their inbox alone has reached the target size.

The essence of this mistake is that more-consistent data (the declared winner) is influenced by less-consistent data (the inbox size). This inappropriate influence means the developers of `declare_winner()` would have to add complex code to ensure its assumptions hold under weak consistency guarantees, even though `declare_winner()` does not access weakly consistent data itself.

The issue of weakly consistent data influencing strongly consistent computations is fundamental to the semantics of consistency. Even within a linearizable transaction, the influence of weakly consistent data on program control flow can effectively weaken the isolation level of the entire transaction. MixT uses information flow analysis to flag such influences at compile time, disallowing this example code. As discussed in Section 6, MixT also allows intentional weakening of this restriction.

3 MixT Transaction Language

We solve the problems introduced in the previous section with MixT, a new domain-specific language (DSL). To support a variety of underlying stores in a uniform way, including key-value stores, databases, and file systems, MixT offers a high-level embedded transaction language that is straightforward to adapt to new stores.

3.1 MixT Language Syntax

Figure 3 gives the surface syntax of the MixT language. Because MixT is embedded in C++, its syntax and semantics, though different from those of its host language, are designed to be unsurprising to C++ programmers.

MixT is relatively expressive; for example, it has control structures like conditionals and loops. Despite supporting real control structures, MixT transactions are fully atomic when the underlying stores support atomic transactions. In

$$\begin{aligned}
& x \in \text{Var} \quad f \in \text{Operation} \\
& \oplus \in \text{Binop} \quad \ominus \in \text{Unop} \\
(\text{Location}) \ m &::= x \mid *e \mid e.x \mid e \rightarrow x \\
(\text{Expr}) \ e &::= m \mid e_1 \oplus e_2 \mid \ominus e \mid e_0.f(e_1, \dots, e_n) \\
(\text{Stmt}) \ s &::= \text{var } x = e \mid m = e \mid \text{return } e \\
& \mid \text{while } (e) \ s \mid \text{if } (e) \ s_1 \text{ else } s_2 \mid \{s_1, \dots, s_n\}
\end{aligned}$$

Figure 3. MixT surface syntax. Certain built-in operations are omitted for clarity of exposition.

particular, all transaction effects become visible at once, and transactions operate against stable snapshots at each store. Like C++, the MixT language has mutable locations, which can be either local variables or fields of objects.

Though they are not shown explicitly in Figure 3, *handles* are a key abstraction of MixT. Handles behave like pointers to remotely stored persistent data; they can be dereferenced to access the underlying data (with the operators $*$ and $\rightarrow$), and they can be aliased by assignment.

Handles also support the invocation of operations on data. Given a handle e_0 to a receiver object, the expression $e_0.f(e_1, \dots, e_n)$ invokes a *custom operation* named f , provided by the underlying store of the receiver.¹ Exactly which operations are supported depends on the store. For example, many stores provide specialized operations for manipulating sets, but even SQL queries can be exposed as operations.

3.2 A MixT Example Program

As an example of MixT code used within a larger program, the message delivery transaction of Section 2.2 is shown in Figure 4. To distinguish MixT code from surrounding C++ code, MixT code is colored **green**, whereas C++ code is **blue**.

Most of this code should look familiar to a C++ programmer; outside the transaction, it merely defines classes that contain library types, such as the MixT library type `RemoteList`, as fields.

At the heart of MixT are transaction blocks, signified by the `mixt_method` declaration. For example, at lines 8–14 is the now-familiar transaction for message delivery, expressed as a method `add_post()` of the C++ class `group`. This method can be invoked from any context without the need to explicitly start a transaction; its parameter `post` is automatically inferred to be a string.

¹ The store may specify whether its parameters should be treated as opaque handles, arbitrary values, or dereferenced handles to other objects on this store. The arguments $e_1, \dots, e_n$ are passed as values by default, except when the store requests otherwise, at which point they will be dereferenced (resulting in a runtime error if these arguments refer to handles on the wrong store).

```

1 class user {
2   Handle<set<string>, causal, supports<insert>> inbox;
3 };

5 class group {
6   RemoteList<Handle<user, causal>, linearizable> users;
7   Handle<Log, eventual, supports<append>> log;

8   mixt_method(add_post) (post) mixt_captures(users, log) (
9     var iterator = users,
10    while (iterator.isValid()) {
11      log.append(iterator->v.inbox.insert(post)),
12      iterator = iterator->next
13    }
14  )
15 };

```

Figure 4. MixT message delivery implementation (§4.2). MixT code (lines 8–14) is colored **green**; C++ code is **blue**.

In this transaction, the expressions `iterator`, `inbox`, `users`, and `log` are all handles for state on remote stores. The type `Handle<T, L, ...>` is the C++ representation of a MixT handle for data of type T , stored at consistency L . An object of this class acts as an opaque representation of a persistent resource. Any supported custom operations appear in the third and following argument positions. For example, `inbox` (line 2) is a set of strings, stored at causal consistency, with a custom operation `insert` for adding new items to the set. It is the job of the causal store to ensure that `insert` operations from different clients are merged with causal consistency.

MixT offers some useful data structures as library types. For example, the type `RemoteList`, used at line 6, is a persistent linked list that stores its spine at a specified consistency level (here, linearizable).

3.3 MixT API

As much as possible, MixT operates as a shim above existing stores, reusing their existing mechanisms for replication and data consistency. It is straightforward to add support for a new store, as long as it offers the necessary functionality; one simply implements three interfaces, `Handle`, `DataStore`, and `TransactionContext`, shown in Figure 5.

The `Handle` interface consists of a simple get/put/check API for accessing underlying data, a set of routines for supporting marshaling, and a set of routines for accessing and using the store from which the `Handle` originated. Much of this functionality can be automatically generated by the MixT libraries at compile time; Figure 5 only includes routines the programmer must implement.

A `DataStore` serves as an entry point to the underlying storage system; it is always associated with a specific consistency label (level) and a specific implementation of

```

class Handle<Type, Label, Operations...> {
    Type get(TransactionContext);
    void put(TransactionContext, Type);
    bool isValid(TransactionContext);
    Type clone(TransactionContext);
};

class DataStore<Label> {
    TransactionContext beginTransaction();
};

class TransactionContext {
    bool commit();
    DataStore store();
};

```

Figure 5. Handle and DataStore interfaces.

```

class CausalStore : public DataStore<causal> {
    template<typename T>
    class CausalObj : CausalHandle<T> {...};
    template<typename T>
    mixt_operation(insert) (CausalObj<set<T>>&, T&) {...}
    ...
};

```

Figure 6. Implementing a causal store in a host C++ program.

Handle. The only requirement from the `DataStore` API is the method `beginTransaction()`, which must create a new transaction represented by a `TransactionContext` object. The `TransactionContext` can be used to commit or abort the transaction interactively, and can be extended to supply store-specific transaction interactions options. A `DataStore` may also implement any number of custom operations, ranging in complexity from creating new remote objects to processing SQL statements.

Figure 6 illustrates how a causal store with a custom operation `insert` can be implemented. Custom operations are declared within classes implementing `DataStore` by using the `mixt_operation` keyword. In this example, the operation `insert` is declared to take a remote `set` and a local `T`, matching the types on which it was invoked in `add_post` (Figure 4). Unlike `mixt_method`, `mixt_operation` does *not* declare a C++ method, and can only be called from within a MixT transaction. Within a transaction, operations dynamically dispatch to the appropriate `Handle` and `DataStore`; to facilitate this dispatch, every `Handle`’s type also includes a static list of operations which its implementation supports.

MixT custom operations provide a method-like syntax for invoking operations directly on handles to remote data, as with `insert` in Figure 4. It would be a mistake, however, to

imagine that they are limited only to “method-like” invocations directly on remote data; they are flexible enough to expose arbitrary database functionality directly to a MixT transaction. For example, one could create a `Handle<DB>`, with matching `mixt_operations` for interfacing directly with the underlying database’s raw API. If the database exposed more stateful functionality, such as locks, a `Handle<DBLock>` could be used to manage each individual lock.

4 Mixed-Consistency Transactions

Section 2 shows that even seemingly trivial code can require the implementer to reason very carefully about the interactions between different consistency levels in the presence of possible transaction aborts. The complexity of this reasoning can easily become overwhelming. MixT tames this complexity by providing semantics for mixed-consistency transactions (§4.1). MixT’s transaction support can provide atomic execution for Section 2.1’s message delivery transaction (§4.5), and its type system will detect the fundamental errors of Section 2.3’s contest (§4.3). A more detailed look at MixT’s transactions comes in Section 5.

4.1 Defining Mixed Consistency

We now address a fundamental question: what are the desired semantics of mixed consistency? We choose the standard approach used for shared-memory consistency [25], in which a consistency model is characterized as a trace property: that is, as the (possibly infinite) set of execution traces that do not violate the consistency model’s guarantees. In principle, we can then verify whether a program execution satisfies a given model by checking whether its trace is in the set.

In mixed-consistency transactions, objects labeled with some consistency model should enjoy at least the guarantees of that model. For example, in a system with both eventual consistency and linearizability, traces involving any subset of objects should adhere at least to eventual consistency, and traces involving only its linearizable objects should respect linearizability. Put another way, an observer who accesses only linearizable objects should be unable to determine that there are any eventually consistent operations in the system.

The strength of consistency models can be characterized in terms of the possible behaviors of programs. The behaviors of the programs form a set of admitted traces T . The meaning of a consistency level ℓ is given by its consistency model, a set of traces $T_\ell \subseteq T$. A model T_ℓ is stronger than a model $T_{\ell'}$ when $T_\ell \subseteq T_{\ell'}$; T_ℓ provides more guarantees than $T_{\ell'}$. All consistency models must include the empty trace. We assume there is a lattice of consistency levels $\mathcal{L}$ ordered by strength. If a consistency level ℓ is stronger than or equal to another, ℓ' , we write $\ell \sqsubseteq \ell'$. Consistency models are ordered by inclusion consistently with the ordering on $\mathcal{L}$: $\ell \sqsubseteq \ell' \iff T_\ell \subseteq T_{\ell'}, T_{\ell \sqcup \ell'} = T_\ell \cup T_{\ell'}, \text{ and } T_{\ell \cap \ell'} = T_\ell \cap T_{\ell'}$.

Each trace $t \in T$ is a sequence of events e . An event e is a 5-tuple (a, o, ℓ, v, S) containing a , the action corresponding to this event; o , the exact memory location or object referenced by this event; ℓ , the consistency level of the store for this event's location; v , a tuple of any values processed by this event; and S , the client session in which this event occurred. Given such an event, we define $\text{consistency}((a, o, \ell, v, S)) = \ell$. For example, the program $x = 4; x = x + 1$, wherein x resides on a store with consistency ℓ , admits the trace “(write, $x, \ell, (4), S$); (read, $x, \ell, (4), S$); (write, $x, \ell, (5), S$)” when executed in session S .

Given a trace t , the events relevant to a given consistency level ℓ are those whose consistency level is at least as strong. We write $t \downarrow \ell$ to denote the trace containing such events:

Definition 4.1 (Trace projection).

$$t \downarrow \ell = [e \mid e \in t \wedge \text{consistency}(e) \sqsupseteq \ell]$$

Definition 4.2 (Mixed consistency). A trace t exhibits *mixed consistency* if it satisfies every consistency model T_ℓ when projected onto that consistency level:

$$\forall \ell, t \downarrow \ell \in T_\ell$$

This definition is sensible even when working with incomparable consistency models; because consistency models form a lattice [51], there is always some minimum consistency model onto which all events can be projected.

Definition 4.2 can also be adapted to transaction isolation levels [3] by considering traces containing explicit events that begin and end transactions. A full formalization is found in the technical report [44].

4.2 Noninterference for Mixed Consistency

In Section 4.1, we proposed a definition for *mixed consistency* based on the approach used for shared-memory consistency. But this approach can hide influence: common consistency models, expressed in terms of reads and writes to shared registers, are not strong enough to capture *why* each read or write occurs. To capture this influence directly, we look to *noninterference*, a semantic property common in the security and privacy literature. Noninterference describes programs as secure if, when given a policy lattice of security labels, program behavior at one point in the policy lattice cannot influence behavior at levels that are lower in the lattice or incomparable [21, 47]. In particular, when using noninterference to enforce privacy or confidentiality, two runs of a program that differ only in secret inputs should have identical publicly observable behavior. Noninterference is the correctness condition normally associated with information flow (Section 4.3).

We start by taking this traditional approach, replacing secret with “weakly consistent” and public with “strongly consistent;” in other words, we determine if any “weakly consistent” data can influence any “strongly consistent” data

by comparing the possible runs of transactions. We cannot simply compare pairs of runs, however, because systems built using MixT are inherently concurrent and nondeterministic; two runs may differ simply as a result of acceptable nondeterminism. We instead consider *sets* of possible runs generated by keeping the deterministic program inputs fixed, but varying the nondeterministic choices made by the program. We say that weakly consistent data has an improper influence in this program if varying weakly consistent data introduces new strongly consistent data into the set. Put another way, varying weakly consistent data should only affect strongly consistent values in ways already permitted by the inherent nondeterminism of the system. This *possibilistic* notion of information flow is called generalized noninterference [41].

Possibilistic security has been shown to be problematic in its original setting of confidentiality, because information can be leaked via refinement [52, 60]. In the context of consistency and other integrity-like properties, it does not seem to be a major concern [38].

4.3 Consistency as Information Flow

To enforce generalized noninterference, we treat consistency as a form of information-flow integrity [4] and use an information-flow type system [47] to outlaw bad programs. Previous work [52] has shown that generalized noninterference is soundly enforceable using this style of security type system. In such a type system, values are associated with a label drawn from a lattice, which in this case is a lattice of consistency levels. The strongest possible consistency is the lowest point in the lattice, denoted $\perp$, and the weakest consistency is $\top$. To enforce consistency, information should not be influenced by other information whose consistency is not at least as strong. Therefore, as in other work on information flow, legal information flow is upward in the lattice.

In the case of the buggy contest in Section 2.3, the transaction creates a banned information flow from the inbox size (weak) to the `declare_winner()` operation (strong). In information-flow terms, this is an implicit flow [47]. The type system of MixT (Section 5.2) statically catches invalid flows, whether implicit or explicit, and rejects unsafe transactions.

4.4 Transaction Splitting

We now turn to the difficult task of implementing noninterfering transactions against multiple backing databases. Consider again the message delivery code in Figure 4. This code is noninterferent and is therefore safe in principle, but because it involves three different consistency levels, it is nonetheless quite difficult to implement, as discussed in Section 2.2.

MixT implements mixed-consistency transactions like this one by automatically splitting their code into a single sub-transaction per involved store. A key insight is that safe splitting is always possible because information flow restrictions prevent weakly consistent data from affecting strongly

consistent data either directly or indirectly within a transaction. Hence, transactions can be split so that their stronger-consistency parts are executed earlier. This allows each sub-transaction to be safely re-executed in the case of a transaction abort, avoiding the pitfalls inherent to partitioning data across systems outlined in Section 2.2. This splitting does not automatically preserve atomicity, the subject of Section 4.5.

In general, a *split transaction* consists of a sequence of syntactically separate transaction phases. For each consistency level in the transaction, there is a single phase for all operations with that consistency level. MixT determines which data are communicated between phases, preserving only the information necessary to execute subsequent phases.

For example, the message delivery transaction is split into linearizable, causal, and eventual phases (in order of decreasing strength of consistency guarantees), corresponding to the consistency levels used by the transaction.

The most challenging aspect of transaction splitting is the treatment of loops, which makes splitting quite different than in previous work on automatic transaction splitting [10, 61]. Like all expressions, each loop's condition must be evaluated within a single phase, but the body of the loop might contain statements that execute in different phases. A loop spanning multiple consistency levels, such as in the message delivery transaction, must therefore be re-executed for each consistency level.

The information-flow type system ensures that all statements that affect the loop's condition occur at the first and strongest phase in which the loop appears. In this first phase, MixT explicitly records the results of each conditional test for the loop, replaying them during the loop execution in subsequent phases. For more detail about this process and a worked example, see Section 5.3.

4.5 Whole-Transaction Atomicity

Static transaction splitting produces a single sub-transaction per underlying store, allowing us to inherit the guarantees of isolation and atomicity provided for all operations on that store. Splitting does not, however, guarantee atomicity for the entire transaction, since commits to stronger stores happen before commits to weaker ones. To ensure atomicity, MixT programs must be prevented from observing the effects of partially committed transactions. When atomicity is guaranteed by at most one of the stores to which a transaction writes, no extra machinery is needed. However, for the rare transaction that writes to multiple atomic stores, we introduce *witnesses*, which lock affected objects.

During each phase's execution, MixT creates a special *write witness* object for each mutation, indicating that a lock has been acquired on the object being mutated. At the end of each phase, MixT creates a single *commit witness*, a special object which indicates that all locks acquired during this transaction have been released. Only one witness is produced

```
(Location)  $m ::= x \mid m.x$ 
(Expr)  $e ::= m \mid x_1 \oplus x_2 \mid \ominus x \mid x_0.f(x_1, \dots, x_n)$ 
(Stmt)  $s ::= \text{var } x = e \text{ in } s \mid \text{remote } x = e \text{ in } s$ 
       $\mid m = x \mid \text{return } x \mid \text{while } (x) s$ 
       $\mid \text{if } (x) s_1 \text{ else } s_2 \mid \{s_1, \dots, s_n\}$ 
```

Figure 7. MixT flattened syntax.

per transaction, but a copy of it is sent to every store on which writes were performed. If a MixT transaction encounters a write witness, it must suspend execution until it encounters the corresponding commit witness.

The witness mechanism ties together phases of split transactions across mutually unaware systems. By creating an explicit object during each transaction and blocking future progress until it has appeared, we guarantee atomicity; the full transaction will be visible to all future transactions.

The witness mechanism should impose relatively little overhead because its use should be rare. Further, several optimizations (Section 7.1) can reduce its overhead. The performance evaluation (Section 8) shows that a complex MixT program can achieve reasonable throughput even when witnesses are used. We revisit witnesses in more detail in Section 5.4; formal arguments regarding the correctness of witnesses can be found in the technical report [44].

5 Formalizing the MixT Language

5.1 Desugared Language

To facilitate transaction splitting, MixT's surface syntax is translated to a "flattened" language whose syntax appears in Figure 7. There are a few notable changes from the surface language. All expressions are flattened by the compiler using standard techniques [48]. The pointer-like syntax $*e$ and $e \rightarrow x$ is replaced by the ability to declare *remote* variables bound to handles. Semantically, *remote* variables directly correspond to the referenced location on an underlying store. Updates to these variables are reflected at the store, and uses of these variables query the store directly for their value. Unlike in the surface language, both *var* and *remote* introduce explicit scopes for their bindings.

5.2 Statically Checking Consistency Labels

Consistency is enforced in MixT by statically checking information flow using a largely standard type system for static information flow [47].

Figure 8 gives selected consistency typing rules for the language. Ordinary rules for typing judgments $\Gamma \vdash e : \tau$ are not presented because they directly use the C++ type system; the presented rules are only for *consistency judgments* $\Delta \mid \Gamma \mid pc \vdash e : \ell$. Environments Δ and Γ keep track of the labels and types of variables, respectively, with local and

$$\begin{array}{c}
\frac{\Delta \mid \Gamma \mid pc \vdash e : \ell_1 \quad \Gamma \vdash e : \tau \quad \Delta, x_V : \ell \mid \Gamma, x_V : \tau \mid pc \vdash s : \ell_2 \quad \ell_1 \sqsubseteq \ell \quad x \notin \Gamma \quad x \notin \Delta}{\Delta \mid \Gamma \mid pc \vdash \text{var } x = e \text{ in } s : \ell} \\
\\
\frac{\Gamma \vdash e : \text{Handle}(\tau, \ell_1) \quad \Delta \mid \Gamma \mid pc \vdash e : \ell_2 \quad \Delta, x_R : \ell \mid \Gamma, x_R : \tau \mid pc \vdash s : \ell' \quad \ell_1 \sqcup \ell_2 \sqsubseteq \ell \quad x \notin \Gamma \quad x \notin \Delta}{\Delta \mid \Gamma \mid pc \vdash \text{remote } x = e \text{ in } s : \ell} \\
\\
\text{REMOTE-READ} \\
\frac{pc \sqsubseteq \ell \quad \Delta, x_R : \ell \mid \Gamma \mid pc \vdash x : \ell}{\Delta, x_V : \ell \mid \Gamma \mid pc \vdash x : \ell} \quad \Delta, x_V : \ell \mid \Gamma \mid pc \vdash x : \ell \\
\\
\frac{\Delta \mid \Gamma \mid pc \vdash e : \ell \quad \Delta \mid \Gamma \mid pc \sqcup \ell \vdash s : \ell'}{\Delta \mid \Gamma \mid pc \vdash \text{while } (e) s : \ell}
\end{array}$$

Figure 8. Selected consistency typing rules for MixT. The labeled REMOTE-READ rule is unusual.

remote variables distinguished lexically by subscripts V and R . The label pc (for *program counter*) bounds the consistency of control flow.

The rules assign each statement and expression a consistency label ℓ that reflects the weakest consistency of any information used to compute it. The label on statements, used during transaction splitting, is derived directly from subexpressions and is unaffected by substatements. During static checking, consistency originates from the consistency labels on handles, which derive from their stores. Variables captured from the environment outside of the transaction are labeled with the strongest ($\perp$) consistency; all other labels are automatically inferred from the transaction code.

One non-standard aspect of the rules is that all accesses to `remote`-bound variables are treated as effectful, requiring the same pc consistency to read a `remote` location as to write it (REMOTE-READ). This restriction is imposed for correct transaction splitting, to enforce the necessary condition that all remote operations at a single consistency level execute before any remote operations at a weaker level.

Omitted from Figure 8 are rules governing assignment, operations (treated identically to assignments), and implicit flows; these follow standard conventions for static information flow [47] and can be found in the technical report [44]. Also omitted is the rule governing endorsement, discussed in Section 6.

5.3 Transaction Splitting

Figure 9 gives selected rules for splitting transactions into phases based on consistency labels. The translation $\llbracket \cdot \rrbracket_\ell$ generates the code for the transaction phase at consistency level ℓ .

(Expr) $e ::= \dots \mid \text{rand}() \mid \text{peek}(x)$
 (Stmt) $s ::= \dots \mid \text{release_all}(n)$
 $\mid \text{acquire}(x, n, \ell_1, \dots, \ell_n)$
 $\mid \text{advance}(x) \mid \text{advance remote}(x)$

$$\begin{array}{c}
\llbracket \text{remote } x = e \text{ in } s : \ell \rrbracket_\ell \triangleq \text{remote } x = e \text{ in } \llbracket s \rrbracket_\ell \\
\\
\frac{\ell \not\sqsubseteq \ell'}{\llbracket \text{remote } x = e \text{ in } s : \ell \rrbracket_{\ell'} \triangleq \llbracket s \rrbracket_{\ell'}} \\
\\
\frac{\ell \sqsubseteq \ell'}{\llbracket \text{remote } x = e \text{ in } s : \ell \rrbracket_{\ell'} \triangleq \{\text{advance binding}(x), \llbracket s \rrbracket_{\ell'}\}} \\
\\
\llbracket \text{while } (e) \text{ stmt} : \ell \rrbracket_\ell \triangleq \text{while } (e) \llbracket \text{stmt} \rrbracket_\ell \\
\\
\frac{\ell \not\sqsubseteq \ell'}{\llbracket \text{while } (e) \text{ stmt} : \ell \rrbracket_{\ell'} \triangleq \{\}} \quad \frac{\ell \neq \ell' \quad \ell \sqsubseteq \ell'}{\llbracket x = e : \ell \rrbracket_{\ell'} \triangleq \text{advance}(x)} \\
\\
\frac{\ell \neq \ell' \quad \ell \sqsubseteq \ell'}{\llbracket x : \ell \rrbracket_{\ell'} \triangleq \text{peek}(x)} \quad \llbracket x : \ell \rrbracket_\ell \triangleq x
\end{array}$$

Figure 9. Selected transaction splitting rules.

```

var iterator = users, // Phase: strict   serializability
var loopindex = iterator.isValid(),
while (loopindex) {
  loopindex = iterator.isValid(),
  var temporary0 = iterator->v,
  iterator = iterator->next
}

advance(loopindex), // Phase: causal consistency
while (loopindex) {
  advance(loopindex),
  advance(temporary0),
  var temporary1 = peek(temporary0).inbox.insert(post)
}

advance(loopindex), // Phase: eventual consistency
while (loopindex) {
  advance(loopindex),
  advance(temporary1),
  logger.log(peek(temporary1))
}

```

Figure 10. The message delivery transaction after splitting, lifted back to the surface syntax.

Recall that each statement in the flattened language is associated with exactly one consistency level. Intuitively, transaction splitting preserves a statement in phase ℓ when the statement's label matches ℓ and otherwise omits it from the phase. However, statements associated with a nested

```

class DataStore<Label> {
    ...
    bool exists(Name name);
    Handle<...> existingObject(Name name);
    Handle<...> newObject(Name name);
};

```

Figure 11. Enhanced DataStore interface for witnesses.

scope, such as `while` and `var`, may execute their contained statements in a different phase.

Once some variable x is introduced in a phase, it may be used—but not assigned—in any later phase. During the phase in which x is introduced, every binding and assignment to x is stored in an implicit iterator. During subsequent phases, this iterator is used to replay x 's previous values. In these subsequent phases, uses of x are replaced with `peek( $x$ )` expressions, which return the current value of x 's implicit iterator. Mutations to x are replaced with `advance( $x$ )`, which advances x 's implicit iterator. Remote-bound variables require the additional `advance binding( $x$ )` construct. Recall that the `remote  $x = e$` construct binds x as an alias to the remote state described by e . If this binding appears in a loop, then the value of e may shift, causing x to be bound to a series of remote locations. The `advance binding( $x$ )` statement cycles through these, while the `advance( $x$ )` statement cycles through assignments to the remote object itself. These and other syntactic extensions required by transaction splitting are shown in Figure 9.

For example, the split transaction generated from the message delivery transaction contains three non-local phases, one for each distinct consistency level used, as shown in Figure 10 (for clarity, the code is presented in the surface syntax). Updates to the freshly generated variables `loopindex`, `temporary1`, and `temporary0` are logged; the expression `peek( $x$ )` accesses an iterator over the previous values of x , and the expression `advance( $x$ )` advances this iterator. Neither of the original variables `iterator` and `users` is necessary for any subsequent phase, so they are discarded upon successful commit of the first phase.

5.4 Atomicity for Split Transactions

A user may choose to request full atomicity for transactions that write to multiple stores. In order to ensure these writes are fully atomic, MixT employs the witness mechanism described in Section 4.5. Write witnesses correspond to lock-acquire events, while commit witnesses correspond to lock-release events. Both are represented by objects written directly to underlying storage. Write witnesses contain the locations of all corresponding commit witnesses and a list of all consistency levels involved in the transaction. Commit witnesses are empty objects. We extend the requirements on underlying storage to include a key-value API through

which witnesses can be named (Figure 11). These APIs take the form of type constraints; the return and parameter types must support certain operations, but do not need to be a precise MixT-specified type.

After transaction splitting, the MixT compiler augments the split transaction with code to create witnesses. First, the transaction generates a new variable `wit`, which we will use as the name of our commit witness. To avoid collisions, this name is selected randomly from a reserved 63-bit namespace. Next, it inserts the statement `acquire( $x$ ,  $wit$ ,  $phases$ )` before all mutative operations. This statement creates a write witness, writing it alongside x . At the end of the phase, the compiler appends `release_all( $wit$ )`, which writes the commit witness itself.

The reasoning behind this transformation is simple: each call to `acquire( $x$ ,  $wit$ , ...)` acquires a logical lock on x , which is released by the corresponding call to `release_all( $wit$ )`. Any transaction which observes the “lock acquire” event (`acquire`) must now wait for the corresponding “lock release” event (`release_all`) before proceeding at each participating phase.

At run time, whenever MixT attempts to read a `remote`-bound value, it also reads that value's potential witness location, checking for a write witness to some transaction which wrote this value. If it finds one, it adds the discovered commit witness's location to a *witness worklist* for each phase listed in the write witness.

Before MixT begins executing a phase p , it first iterates through p 's witness worklist. For each commit witness in the worklist, MixT polls p 's store until the commit witness becomes available, and then removes it from the worklist. As described in Section 7.2, this polling loop occurs on the remote store itself.

Once the commit witness has appeared on all replicas, it may be safely removed from the store; the specifics of this process are found in Section 7.1.

Note that using witnesses does not cause the resulting transaction to become fully serializable; if a weak consistency level allows stale reads, then stale reads can still occur during the weak phase of a mixed-consistency transaction. A mixed transaction might fail to read values committed by a previous fully-weak transaction. Even with witnesses, the code in Figure 2 remains incorrect. Also, atomicity is only guaranteed when the underlying store provides it; consistency levels which do not have a useful notion of atomicity—for example, eventual consistency—therefore do not employ write or commit witnesses.

5.5 Satisfying Mixed Consistency

Now that we have reviewed the mechanisms of MixT in detail, we return to the question of *mixed consistency* itself; how do we know that MixT transactions satisfy definition 4.2?

If the consistency model, as is traditional, includes only information about independent reads and writes made to

```

if ((a.inbox.strong_read().size() >= 1000000 &&
    b.inbox.strong_read().size() < 1000000)
    .endorse(strong)) {
    a.declare_winner()
} else
if ((a.inbox.strong_read().size() < 1000000 &&
    b.inbox.strong_read().size() >= 1000000)
    .endorse(strong)) {
    b.declare_winner()
}

```

Figure 12. Strongly consistent contest logic with endorsement.

individual memory locations then it is easy to see that MixT satisfies mixed consistency. Because MixT associates each data object or memory location with a single consistency model, all reads of some value from a memory location at some consistency level ℓ must be paired with a write to that location at level ℓ . Thus, for all reads in a trace, the projection operator ($\downarrow$) also preserves all matching writes in that trace. In this sort of consistency model, the projection operator simply selects sets of memory locations.

6 Upgrading Consistency

6.1 Semantics and Motivation

As described up to this point, MixT transactions maintain a strict separation between consistency levels; it is impossible to use a weakly consistent value to influence a strongly consistent operation. This rigid separation provides strong semantic guarantees, but can be limiting.

As an example, consider the mail-delivery example from Figure 2. During the contest, mail is delivered with causal consistency; users who view their inboxes may see a slightly stale view of the mail they’ve received—an acceptable semantics given the already variable latency of mail delivery itself. The system, however, needs to perform a strongly consistent read to determine a winner once the contest has finished. Let’s imagine that the store supports such an operation, and exposes it to MixT via the name `strong_read`.

In MixT, the result of any expression can be declared to have an arbitrary consistency via the built-in operation `endorse(label)`. It behaves as a type-cast; the MixT compiler makes no effort to ensure that endorsements are used appropriately, as their validity often depends on complex system properties not visible to the compiler. Like all of weak consistency, endorsements must be used with care.

Using our hypothetical `strong_read` operation and MixT’s `endorse` keyword, we can fix our transaction (Figure 12). This code is correct, but runs all operations at a strong consistency level—exactly what we were trying to avoid. To improve performance, we can guard each strongly consistent read with a preliminary weakly consistent test, restoring

```

if (((a.inbox.size() >= 1000000 &&
    b.inbox.size() < 1000000)
    && (a.inbox.strong_read().size() >= 1000000 &&
    b.inbox.strong_read().size() < 1000000))
    .endorse(strong)) {
    a.declare_winner()
} else
if (((a.inbox.size() < 1000000 &&
    b.inbox.size() >= 1000000)
    && (a.inbox.strong_read().size() < 1000000 &&
    b.inbox.strong_read().size() >= 1000000))
    .endorse(strong)){
    b.declare_winner()
}

```

Figure 13. Efficient contest logic with endorsement. The programmer has introduced an additional check involving a rare strong read for when the contest is believed to be over. We also must endorse the enclosing conditional, as it still creates an indirect flow to `declare_winner`.

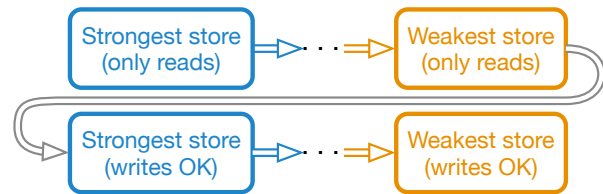


Figure 14. Transaction phases with endorsement.

causal execution for most transactions while ensuring the declaration of a winner is still guarded by a strongly consistent condition (Figure 13). The resulting consistency of the strong `declare_winner()` operation is no longer separable from the causally consistent read; we have accepted the possibility that our winner may be declared late.

6.2 Compiling Endorsements

Because of transaction splitting, endorsement is not straightforward. It fundamentally involves running weaker commands before stronger ones — and thus requires more than one phase per underlying store. But naively adding additional phases will not provide acceptable semantics; having one phase per underlying store is key to MixT’s isolation guarantees.

We address these concerns with two mechanisms: *read-only phases* and read witnesses. As in other information-flow languages, endorsement is indicated by annotating the endorsed expression. The compiler separates the transaction into two parts: the pre-endorse part and the post-endorse part. To ensure atomicity, the pre-endorse part of the transaction is checked to ensure that it contains no writes. The code is then split into phases in the usual way, except that an artificial “pre-endorse label” is first joined with all labels in

the pre-endorse code so that pre-endorse code appears to the compiler to have stronger consistency than all post-endorse code. This process is depicted in Figure 14. If full isolation is required, read witnesses may be employed. Read witnesses are dual to write witnesses; the creation of a read witness accompanies every read, and checking for read witnesses occurs before every write. More details on read witnesses can be found in the technical report [44]. An optimization which uses read validation in place of read witnesses (as in optimistic concurrency control) is also possible [34].

7 Implementation

MixT has been implemented as four separate C++17 components, numbering almost 30,000 lines in total: the transactions language compiler (10k), the core library (2.8k), the tracking mechanism (1k), the Postgres implementation (1.4k), and support utilities (14k). The current implementation supports an unbounded number of backing stores in a single application.

To evaluate MixT, we also developed several sample backing stores, operating either in-memory or based on PostgreSQL 9.4. These interfaces expose a selected set of prepared statements as custom operations and are designed to provide linearizability (with strict serializability), causal consistency (with snapshot isolation), and eventual consistency (with read-uncommitted isolation.)

7.1 Efficiency Optimizations

Recall that mixed-consistency transactions use witnesses to ensure atomicity, wherein an extra read operation accompanies each stated read, and an extra write operation accompanies each transaction. As described so far, this mechanism would frequently encounter stale values, harming performance for no gain in safety. Additionally, commit witnesses would slowly accumulate on the store, wasting storage. Ideally, we would be able to determine whether a value was stable across an entire store, and therefore did not require such defensive tracking behavior. To accomplish this we observe that, in order to maintain its own guarantees regarding operation order, our non-compositional store likely has some notion of a timestamp or version number already available for internal use. This assumption proves true in practice: systems such as COPS, Eiger, 2-master PostgreSQL, Bolt-on, TARDIS, HBase, and Mongo (via Vermongo) all either use these vector clocks directly or are easily modified to employ them [2, 13, 39, 40, 46]. We enhanced our `Handle` API to allow client stores to expose this notion of time to MixT through an optional `timestamp` method. We augment read and write witnesses to include the “current time” of transaction commit, and provide a lightweight TCP protocol through which backing stores can notify MixT clients of the most recent version number which is guaranteed to have reached the entire store. If the ability to access an accurate

transaction commit time from within a transaction does not exist, commit witnesses can be augmented to include the addresses of all read and write witnesses created during the transaction. Leveraging this additional information, MixT avoids generating witnesses when the objects involved are already widely available, and can safely remove stale commit witnesses.

7.2 Remote Execution in MixT

With MixT’s ability to split transactions into phases comes the opportunity to distribute the transaction code itself. By deploying a lightweight worker process alongside existing backing stores, MixT application programmers can run transaction phases directly at stores, incurring only a single round trip to establish each phase and collect its results – and allowing all witness checks to be carried out locally. In fact, this decision—to ship transaction code directly to the storage system—has become increasingly popular among high-performance data storage systems and is central to some modern databases [17, 18, 30, 53]. MixT’s approach to remote execution is straightforward. We assume that each application manages its own lightweight worker at the storage location; we leave the task of ensuring code is up-to-date to the MixT application programmer.

7.3 MixT Compiler Implementation

We implemented MixT as a domain-specific language embedded into modern C++. MixT is written in pure C++17, and can be compiled using any C++17-compliant compiler². Our entire compiler is written in `constexpr` C++ [43], allowing it to run during the “template expansion” step of compilation of the surrounding C++ code. Specifically, `mixt_<keyword>` macros convert their arguments into a compile-time string, which is then parsed and compiled by our compiler. In order to link names within the transactions language to native C++ objects, the macros capture both the type of their arguments and their string representations, using these during the transaction compilation. All compilation, including transaction splitting, is accomplished alongside C++ compilation; none is deferred until runtime. Transactions are compiled to a set of inlined, statically bound functions which are invoked from a single point in code, allowing the C++ compiler to optimize away all function-call overhead, producing machine code quite close to the syntax specified by the transaction. This approach allows MixT to support arbitrary syntax, semantics, and type systems, without requiring an external compiler or preprocessor, and without adding unnecessary run-time overhead. We are not bound to the syntax, semantics or keywords of C++; MixT’s similarity to C++ is a conscious design choice. We follow the language-as-a-library paradigm: as

²The MixT compiler is tested under `≥clang-3.9` and `≥g++-7.1`; certain syntax extensions require `-fconstexpr`

MixT effectively adds extra phases to existing C++ compilation, to use MixT in existing C++ projects all one must do is `#include` the MixT header files.

8 Evaluation

We use the MixT implementation to model an intended application domain: user-facing application servers that share one linearizable and one causally consistent underlying storage system, where application servers are geographically close to only the causal replica they are using. We believe this closely mirrors reality. Weakly consistent storage servers can be relatively close to application servers because they are able to withstand high latencies during replication and can therefore be separated geographically; linearizable data stores are typically housed within a single data center because latencies encountered during replication have an outsized impact on overall performance.

In this setting, we explore several key questions regarding the performance of MixT:

- Do mixed-consistency transactions, as promised, offer better performance than running similarly atomic transactions with strong consistency?
- What overhead is added by the witness mechanism used to preserve consistency guarantees when non-compositional consistency levels are combined?
- On what workloads does this mechanism work well? What is the performance impact of different mixtures of mixed and pure transactions?

8.1 Experimental Setup

To measure the performance of MixT, we simulate a geo-replicated application. In our setup, logically separate application servers each maintain connections to causally consistent and serializable databases. Connections to the serializable database experience a round-trip latency of $85\text{ms} \pm 10\text{ms}$; connections to causally consistent databases experience a round-trip latency of 1ms. Latency to the causal system was set by measuring ping times between an Internet2-connected university and its nearest data center; latency to the linearizable system was set by measuring ping times between Internet2-connected universities on the east and west coasts of the United States. All latency simulations are provided by the netem kernel module on Linux 3.17. We employ three separate physical machines: one hosting all clients, one hosting the causal store, and one hosting the linearizable store.

For driving load to application servers, we adopted a semi-open world model, with delay between events following an exponential distribution. We increase load by increasing the number of MixT clients, not by increasing the rate of events issued by each client. Our causal and linearizable stores are both backed by PostgreSQL. To implement causal consistency, we created four replicas of data within the database,

each associated with a four-entry vector clock version number. More details on the experimental setup are available in the technical report [44].

8.2 Benchmarks

We could find no existing benchmarks for mixed-consistency transactional systems. Instead, we developed two new benchmarks intended to represent the emerging mixed-consistency landscape. The first is a simple microbenchmark based on incrementing integral counters. It features read-only transactions that fetch the value at a counter, and read-write transactions which increment that value. Objects referenced during read operations are selected from a Zipf distribution over 400,000 names; objects referenced during write operations are selected from a uniform distribution over the same names. These objects are duplicated on both a linearizable and casual store. In this benchmark, clients randomly move between causal mode, where all transactions are causally-consistent, and a linearizable mode, where all transactions are linearizable, with a fixed probability. We extend this benchmark to involve mixed-consistency transactions in the next section.

The second benchmark is the Message Groups example discussed in Section 2.2. This benchmark features four more-complex transactions: message delivery (Figure 4), user creation, inbox checking, and group joining. User creation and inbox checking are causally consistent, while message posting and group joining are mixed-consistency transactions. Each client is assigned a range of inboxes and groups from which it selects uniformly at random.

8.3 Counter Results

The counters benchmark offers several tuning parameters for exploring the space of workloads. As copies of our complete set of objects exist on both a causal and linearizable store, we can fine-tune both the mixture of causal and linearizable operations and the combination of reads and writes in our tests.

Speedup Relative to Linearizability The most important question for MixT performance is whether mixed-consistency transactions offer a speedup compared to the simple alternative of running transactions entirely with linearizability. Figure 15 shows that, indeed, mixing causal and serializable operations considerably increases maximum throughput.

Because of the high latencies incurred by serializable transactions, increasing the causal percentage of overall operations yields significant performance improvements. These benefits level off at about 80% causal in our tests; at this point, the causal storage system becomes overloaded, limiting the benefits of lower latency.

Overhead of Witnesses One concern about MixT might be the overhead introduced by witnesses. We modify our

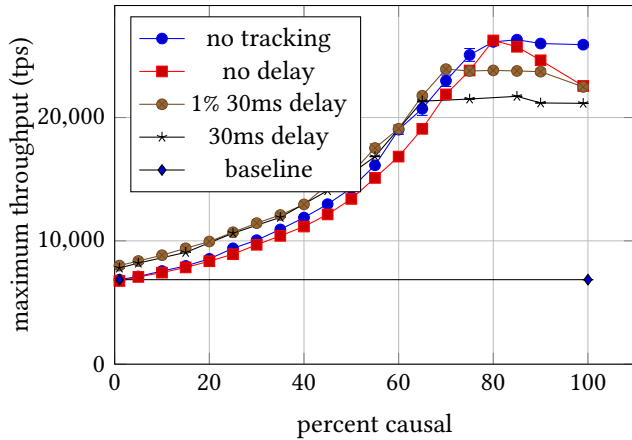


Figure 15. Maximum throughput as a function of linearizable mix for a 70% read workload. The blue (top circle) series shows maximum achievable throughput in transactions per second (tps) without witnesses; the remaining series shows full witness tracking with progressive artificial latency. The solid black line marks 0% causal without tracking (also the leftmost blue point), which serves as a baseline.

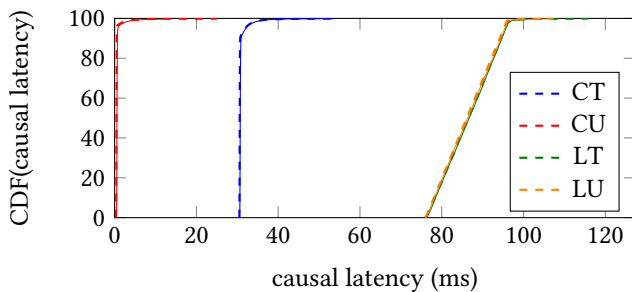


Figure 16. CDF plots for operation latency. C: Causal, L: Linearizable, T: Tracked, U: Untracked. Dashed lines: reads, solid lines: writes. All linearizable lines appear atop each other on the right.

simple counter increment test to explicitly include both linearizable and causal phases in transactions which follow a consistency mode switch, and to force witness generation in these transactions even if they would not normally require it. The rationale for this is based on the non-compositionality of causal consistency, discussed in our technical report [44]. We additionally modify our experimental setup to simulate latency of replication, first forcing approximately 1% of causal witness verifications to delay for 30ms, then explicitly delaying all causal witness verification requests by 30ms. As seen in Figure 15, the witness mechanism has a noticeable impact mostly above 60% causal, with a maximum slowdown of approximately 10%: well above the performance possible were the entire transaction mix to remain linearizable. Further demonstrating that round-trip time is paramount, read-only

Transaction	Throughput (tps)	R	W	C	L
Check inbox	10,626 $\pm$ 15	6	0	✓	×
Join group	5,430 $\pm$ 30	2	1	✓	✓
Deliver message	3,313 $\pm$ 4	6	3	✓	✓
Create user	972 $\pm$ 19	5	2	✓	✓

Figure 17. Maximum throughputs for Message Groups, with standard error. The rightmost four columns give the number of reads (R) and writes (W) and indicate whether the transaction involves a causal (C) or linearizable (L) phase.

transactions achieved only 20% higher maximum throughput than read-write transactions.

Latency Witnesses do affect latency, especially because of design decisions made for backward compatibility. Figure 16 shows this effect. Latencies are presented as a CDF collected from the system running at 60% of maximum throughput, in a configuration in which 75% of all operations are reads and 75% of all operations use the causal store. To see the worst-case impact of witnesses, we ran this test twice: once with witnesses enabled and a forced 30ms delay (“tracked”), and once without (“untracked”). The red (leftmost) and orange (rightmost) lines on this graph measure performance without witnesses; the green (also rightmost) and blue (middle) lines represent performance with witnesses. The forced 30ms delay on witnesses is quite clear for causal operations, but there is almost no other overhead. On the other hand, the impact of witnesses on linearizable operations is negligible; as witnesses incur no replication delay in the linearizable store, they never delay linearizable phases.

8.4 Message Groups Results

To evaluate the running Message Groups example, we use a configuration with 40,000 groups, each of which contains a single distinct user. Each of these 40,000 users has a single message in their inbox. We disable message logging, eliminating all eventually-consistent phases and leaving only causal and linearizable phases. We first run each Message Groups transaction in isolation against this initial configuration, establishing an average maximum throughput over at least 3 runs (Figure 17). This table lists the average maximum throughput for each transaction in isolation, along with the number of read and write operations executed during these transactions. For all transactions, we report the number of operations executed when in our initial configuration; message delivery and inbox downloading require more operations as the group and inbox sizes grow. The purely causal inbox download transaction benefits from the speed of causal consistency, while the mixed-consistency transactions all achieve reasonable performance despite the overhead of contacting a distant linearizable store.

We also evaluate performance on a mix of transactions: 56% inbox checking, 20% message posting, 18% group joining,

and 6% user creation. We evaluate the system for 3 minutes, slowly increasing the client request rate from 2,000 tps to 5,000 tps. Average maximum throughput over 4 trials was $4,237 \pm 10.5$ tps with an abort rate between .0161% and .0187%. This represents a speedup of 3.5 over a baseline in which all operations execute against the linearizable store (average maximum throughput: $1,228 \pm 15$ tps). As expected, mixed-consistency transactions yield significant speedup.

9 Related Work

Quelea [51] and Disciplined Inconsistency [26] are the work closest to MixT in spirit. Both use user-provided data annotations to infer appropriate consistency levels for operations within a single program. The system then automatically chooses an appropriate consistency model for each operation. The Quelea approach, using Cassandra to store compressed logs of all system events, differs markedly from MixT's approach of transaction partitioning based on static information flow analysis. Disciplined Inconsistency also uses information flow to enforce separation between consistency labels but does not offer any transactional mechanism.

Choosing Consistency Levels Choosing appropriate consistency models for data is a problem orthogonal to our work. Prior work [22, 24, 26, 27, 36, 37, 51] provide languages of constraints to describe data invariants, in turn providing the weakest consistency possible while still satisfying those constraints. Other work [7, 22, 29] aims for formal methods for users to reason about their choice of consistency level and to prove that desired code invariants are satisfied.

Transactions in Weak Geo-Replicated Systems Existing work in the shared memory [15] and distributed systems [1, 16, 28, 40, 42, 50] communities has attempted to provide single-store transactions in the presence of weak consistency guarantees. This prior work focuses on definitions and mechanisms for weak transactions at a single consistency level, and indeed, we rely on the guarantees they provide.

Mixed-Consistency Systems Many existing data stores provide operations with a variety of consistency guarantees [11, 14, 35, 37, 46], but without providing any semantic guarantees across operations. Others provide tools to tune consistency based primarily on performance considerations [9, 54, 58]. Guerraoui et al. [23] define a unique programming model by which programs are first presented with weakly consistent data and may choose to wait for strong data instead. These systems provide neither general transaction mechanisms nor strong semantic guarantees.

Mixed-Consistency Systems with Transactions Previous work [19, 32, 33, 37, 58] focuses on progressively weakening transaction isolation based on a combination of run-time and static analysis, with the aim of enforcing strong consistency. Several papers provide mechanisms for users to choose

transaction isolation levels [7, 29, 56], but do not handle the semantic anomalies involved. A few systems [15, 58] provide distributed transactions at multiple consistency levels, but allow unsafe mixing of consistency levels. Microsoft's new database Cosmos DB is a recent example, providing transactions with a choice of four well-defined consistency levels [20]. Some prior systems do enable programmers to mix transactions of different consistency with strong guarantees [37, 49, 57]. However, this line of work relies on a closed-transaction model wherein the system is aware of all possible transactions any client will run; performance is brittle because changing a single transaction somewhere in the system can significantly affect the performance of unrelated transactions. This work cannot mix consistency within a single transaction, and it focuses on a single store. Nevertheless, these systems could be used by MixT as backing stores.

Enhancing Consistency of Existing Systems Beyond SQL, some existing work has focused on mechanisms that upgrade the consistency guarantees of weakly consistent underlying stores [2, 28, 50]. Indeed, several projects [40, 51] use this approach internally, adding consistency layers atop existing distributed systems like Cassandra.

10 Conclusion

We have introduced a new domain-specific programming language for writing modern geodistributed applications that need to trade off performance and consistency. The mixed-consistency transactions offered by MixT make it possible for programmers to safely combine data from multiple consistency levels in the same transaction, with confidence that weaker data does not corrupt the guarantees of stronger data. Appealingly, this model can be implemented in a backward-compatible way on top of existing stores that offer their own distinct consistency guarantees, without disrupting legacy applications on those stores. The performance results suggest that for geodistributed applications, mixed-consistency transactions enable higher performance by using weaker consistency models selectively and safely.

Acknowledgments

The authors would like to thank Jane Brown, Stephanie DeMane, Jon Sailor, Cody Mello, and Robert Mustacchi for their help determining industry applications. The authors would like to thank Ethan Cecchetti for help with PostgreSQL, Nate Nystrom for discussions about information flow and consistency, Isaac Sheff for extensive consultation, and Tom Magrino, Daoji Huang, Eric Lee, Eston Schweickart, Andrew Hirsch, and Sydney Zink for their editorial assistance.

This research was conducted with Government support under and awarded by DoD, Air Force Office of Scientific Research, National Defense Science and Engineering Graduate (NDSEG) Fellowship, 32 CFR 168a, and by NSF grant 1717554.

References

- [1] Masoud Saeida Ardekani, Pierre Sutra, and Marc Shapiro. 2013. Non-Monotonic Snapshot Isolation: Scalable and Strong Consistency for Geo-Replicated Transactional Systems. In *Reliable Distributed Systems (SRDS), 2013 IEEE 32nd International Symposium on*. IEEE, 163–172.
- [2] Peter Bailis, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. 2013. Bolt-On Causal Consistency. In *ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 761–772.
- [3] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O’Neil, and P. O’Neil. 1995. A Critique of ANSI SQL Isolation Levels. In *ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 1–10.
- [4] K. J. Biba. 1977. *Integrity Considerations for Secure Computer Systems*. Technical Report ESD-TR-76-372. USAF Electronic Systems Division, Bedford, MA. (Also available through National Technical Information Service, Springfield Va., NTIS AD-A039324).
- [5] Eric Brewer. 2010. A Certain Freedom: Thoughts on the CAP Theorem. In *Proceedings of the 29th ACM SIGACT-SIGOPS symposium on Principles of distributed computing*. ACM, 335–335.
- [6] Jane William Skillen Brown. 2017. Personal communication. (Nov. 2017). Google, Inc.
- [7] Lucas Brutschy, Dimitar Dimitrov, Peter Müller, and Martin T. Vechev. 2017. Serializability for Eventual Consistency: Criterion, Analysis, and Applications. In *44th ACM Symp. on Principles of Programming Languages (POPL)*. 458–472.
- [8] Josiah L. Carlson. 2013. *Redis in Action*. Manning Publications Co., Greenwich, CT, USA.
- [9] Shankha Chatterjee and Wojciech Golab. 2017. Self-Tuning Eventually-Consistent Data Stores. In *International Symposium on Stabilization, Safety, and Security of Distributed Systems*. Springer, 78–92.
- [10] Alvin Cheung, Samuel Madden, Owen Arden, and Andrew C. Myers. 2012. Automatic Partitioning of Database Applications. *PVLDB* 5, 11 (Aug. 2012), 1471–1482.
- [11] Brian F Cooper, Raghu Ramakrishnan, Utkarsh Srivastava, Adam Silberstein, Philip Bohannon, Hans-Arno Jacobsen, Nick Puz, Daniel Weaver, and Ramana Yerneni. 2008. PNUTS: Yahoo!’s hosted data serving platform. *Proceedings of the VLDB Endowment* 1, 2 (2008), 1277–1288.
- [12] James C Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, Jeffrey John Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, et al. 2013. Spanner: Google’s globally distributed database. *ACM Transactions on Computer Systems (TOCS)* 31, 3 (2013), 8.
- [13] Natacha Crooks, Youer Pu, Nancy Estrada, Trinabh Gupta, Lorenzo Alvisi, and Allen Clement. 2016. TARDiS: A Branch-and-Merge Approach To Weak Consistency. In *ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 1615–1628.
- [14] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Vossahl, and Werner Vogels. 2007. Dynamo: Amazon’s Highly Available Key–Value Store. In *21st ACM Symp. on Operating System Principles (SOSP)*. 205–220.
- [15] Brijesh Dongol, Radha Jagadeesan, and James Riely. 2018. Transactions in Relaxed Memory Architectures. In *45th ACM Symp. on Principles of Programming Languages (POPL)*. 18:1–18:29.
- [16] Jiaqing Du, Călin Iorgulescu, Amitabha Roy, and Willy Zwaenepoel. 2014. Gentlerain: Cheap and Scalable Causal Consistency with Physical Clocks. In *Proceedings of the ACM Symposium on Cloud Computing*. ACM, 1–13.
- [17] Ayush Dubey, Greg D. Hill, Robert Escriva, and Emin Gün Sirer. 2016. Weaver: A High-Performance, Transactional Graph Database Based on Refinable Timestamps. *Proceedings of the VLDB Endowment* 9, 11 (2016), 852–863.
- [18] Jennie Duggan, Aaron J Elmore, Michael Stonebraker, Magda Balazinska, Bill Howe, Jeremy Kepner, Sam Madden, David Maier, Tim Mattson, and Stan Zdonik. 2015. The BigDAWG Polystore System. *ACM SIGMOD Record* 44, 2 (2015), 11–16.
- [19] Lei Gao, Mike Dahlin, Amol Nayate, Jiandan Zheng, and Arun Iyengar. 2003. Application-Specific Data Replication for Edge Services. In *Proceedings of the 12th international conference on World Wide Web*. ACM, 449–460.
- [20] Mimi Gentz, Aravind Krishna R, Luis Bosquez, Mark McGee, Tyson Nevil, Kris Crider, Yaron Y. Golland, Andy Pasic, and Ji Huang Carol Zeumault. 2017. Welcome to Azure Cosmos DB. <https://docs.microsoft.com/en-us/azure/cosmos-db/introduction>. (2017).
- [21] Joseph A. Goguen and Jose Meseguer. 1982. Security Policies and Security Models. In *IEEE Symp. on Security and Privacy*. 11–20.
- [22] Alexey Gotsman, Hongseok Yang, Carla Ferreira, Mahsa Najafzadeh, and Marc Shapiro. 2016. ‘Cause I’m Strong Enough: Reasoning About Consistency Choices in Distributed Systems. In *43rd ACM Symp. on Principles of Programming Languages (POPL)*. 371–384.
- [23] Rachid Guerraoui, Matej Pavlovic, and Dragos-Adrian Seredinschi. 2016. Incremental Consistency Guarantees for Replicated Objects. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI’16)*. 169–184.
- [24] M. Herlihy. 1991. Wait-Free Synchronization. *ACM Transactions on Programming Languages* 1, 13 (Jan 1991), 124–149.
- [25] M. Herlihy and J. Wing. 1988. *Linearizability: A Correctness Condition for Concurrent Objects*. Technical Report CMU-CS-88-120. Carnegie Mellon University, Pittsburgh, Pa.
- [26] Brandon Holt, James Bornholt, Irene Zhang, Dan Ports, Mark Oskin, and Luis Ceze. 2016. Disciplined Inconsistency with Consistency Types. In *Proceedings of the Seventh ACM Symposium on Cloud Computing*. ACM, 279–293.
- [27] Brandon Holt, Irene Zhang, Dan Ports, Mark Oskin, and Luis Ceze. 2015. Claret: Using Data Types for Highly Concurrent Distributed Transactions. In *1st Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC)*. Article 4, 4:1–4:4 pages.
- [28] Ta-Yuan Hsu and Ajay D. Kshemkalyani. 2018. Value the Recent Past: Approximate Causal Consistency for Partially Replicated Systems. *IEEE Transactions on Parallel and Distributed Systems* 29, 1 (2018), 212–225.
- [29] Gowtham Kaki, Kartik Nagar, Mahsa Nazafzadeh, and Suresh Jaganathan. 2017. Alone Together: Compositional Reasoning and Inference for Weak Isolation. *arXiv preprint arXiv:1710.09844* (2017).
- [30] Robert Kallman, Hideaki Kimura, Jonathan Natkins, Andrew Pavlo, Alexander Rasin, Stanley Zdonik, Evan P. C. Jones, Samuel Madden, Michael Stonebraker, Yang Zhang, John Hugg, and Daniel J. Abadi. 2008. H-Store: A High-Performance, Distributed Main Memory Transaction Processing System. *PVLDB* 1, 2 (Aug. 2008), 1496–1499.
- [31] Rusty Klopheus. 2010. Riak Core: Building Distributed Applications Without Shared State. In *ACM SIGPLAN Commercial Users of Functional Programming (CUFP ’10)*. ACM, New York, NY, USA, Article 14, 1 pages.
- [32] Tim Kraska, Martin Hentschel, Gustavo Alonso, and Donald Kossmann. 2009. Consistency Rationing in the Cloud: Pay Only When it Matters. *Proceedings of the VLDB Endowment* 2, 1 (2009), 253–264.
- [33] Tim Kraska, Gene Pang, Michael J Franklin, Samuel Madden, and Alan Fekete. 2013. MDCC: Multi-Data Center Consistency. In *Proc. 8th ACM European Conference on Computer Systems*. ACM, 113–126.
- [34] H. T. Kung and J. T. Robinson. 1981. On Optimistic Methods for Concurrency Control. *ACM Trans. on Database Systems* 6, 2 (June 1981), 213–226.
- [35] Avinash Lakshman and Prashant Malik. 2010. Cassandra: a decentralized structured storage system. *ACM SIGOPS Operating Systems Review* 44, 2 (2010), 35–40.

- [36] Cheng Li, Joao Leita, Allen Clement, Nuno Preguiça, Rodrigo Rodrigues, and Viktor Vafeiadis. 2014. Automating the Choice of Consistency Levels in Replicated Systems. In *USENIX Annual Technical Conference*.
- [37] Cheng Li, Daniel Porto, Allen Clement, Johannes Gehrke, Nuno Preguiça, and Rodrigo Rodrigues. 2012. Making Geo-Replicated Systems Fast as Possible, Consistent when Necessary. In *10th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*.
- [38] Jed Liu and Andrew C. Myers. 2014. Defining and Enforcing Referential Security. In *3rd Conf. on Principles of Security and Trust (POST)*. 199–219.
- [39] Wyatt Lloyd, Michael J. Freedman, Michael Kaminsky, and David G. Andersen. 2011. Don't settle for eventual: scalable causal consistency for wide-area storage with COPS. In *23rd ACM Symp. on Operating System Principles (SOSP)*. 401–416.
- [40] Wyatt Lloyd, Michael J. Freedman, Michael Kaminsky, and David G. Andersen. 2013. Stronger Semantics for Low-Latency Geo-Replicated Storage. In *10th USENIX Symp. on Networked Systems Design and Implementation (NSDI)*. 313–328.
- [41] Daryl McCullough. 1987. Specifications for Multi-Level Security and a Hook-up Property. In *IEEE Symp. on Security and Privacy*. IEEE Press, 161–161.
- [42] Syed Akbar Mehdi, Cody Little, Natacha Crooks, Lorenzo Alvisi, Nathan Bronson, and Wyatt Lloyd. 2017. I Can't Believe It's Not Causal! Scalable Causal Consistency with No Slowdown Cascades.. In *NSDI*. 453–468.
- [43] Scott Meyers. 2014. *Effective Modern C++: 42 Specific Ways to Improve Your Use of C++ 11 and C++ 14*. O'Reilly Media, Inc..
- [44] Mae Milano and Andrew Myers. 2017. MixT: A Language for Mixing Consistency in Geodistributed Transactions: Technical Report. (2017).
- [45] C. H. Papadimitriou. 1979. The Serializability of Concurrent Database Updates. *J. ACM* 26, 4 (Oct. 1979), 631–653.
- [46] Eelco Plugge, Peter Membrey, and Tim Hawkins. 2010. *The Definitive Guide to MongoDB: The noSQL Database for Cloud and Desktop Computing*. Apress.
- [47] Andrei Sabelfeld and Andrew C. Myers. 2003. Language-Based Information-Flow Security. *IEEE Journal on Selected Areas in Communications* 21, 1 (Jan. 2003), 5–19.
- [48] Amr Sabry and Matthias Felleisen. 1993. Reasoning About Programs in Continuation-Passing Style. *Lisp and Symbolic Computation* 6, 3–4 (Nov. 1993), 289–360.
- [49] Dennis Shasha, Francois Lirbat, Eric Simon, and Patrick Valduriez. 1995. Transaction Chopping: Algorithms and Performance Studies. *ACM Trans. on Database Systems* 20, 3 (Sept. 1995), 325–363.
- [50] Kazuyuki Shudo and Takashi Yaguchi. 2017. Causal Consistency for Data Stores and Applications as They are. *Journal of Information Processing* 25 (2017), 775–782.
- [51] K. C. Sivaramakrishnan, Gowtham Kaki, and Suresh Jagannathan. 2015. Declarative Programming over Eventually Consistent Data Stores. In *36th ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*. 413–424.
- [52] Geoffrey Smith and Dennis Volpano. 1998. Secure Information Flow in a Multi-Threaded Imperative Language. In *25th ACM Symp. on Principles of Programming Languages (POPL)*. 355–364.
- [53] Michael Stonebraker and Ariel Weisberg. 2013. The VoltDB Main Memory DBMS. *IEEE Data Eng. Bull.* 36, 2 (2013), 21–27.
- [54] Douglas B. Terry, Vijayan Prabhakaran, Ramakrishna Kotla, Mahesh Balakrishnan, Marcos K. Aguilera, and Hussam Abu-Libdeh. 2013. Consistency-based service level agreements for cloud storage. In *24th ACM Symp. on Operating System Principles (SOSP)*. 309–324.
- [55] Douglas B. Terry, Marvin M. Theimer, Karin Petersen, Alan J. Demers, and Mike J. Spreitzer. 1995. Managing Update Conflicts in Bayou, a Weakly Connected Replicated Storage System. In *15th ACM Symp. on Operating System Principles (SOSP)*. 172–183.
- [56] Chao Xie, Chunzhi Su, Manos Kapritsos, Yang Wang, Navid Yaghmazadeh, Lorenzo Alvisi, and Prince Mahajan. 2014. Salt: Combining ACID and BASE in a distributed database. In *11th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, Vol. 14. 495–509.
- [57] Chao Xie, Chunzhi Su, Cody Little, Lorenzo Alvisi, Manos Kapritsos, and Yang Wang. 2015. High-Performance ACID via Modular Concurrency Control. In *Proceedings of the 25th Symposium on Operating Systems Principles (SOSP '15)*. ACM, New York, NY, USA, 279–294.
- [58] Yingyi Yang, Yi You, and Bochuan Gu. 2017. A Hierarchical Framework with Consistency Trade-off Strategies for Big Data Management. In *Computational Science and Engineering (CSE) and Embedded and Ubiquitous Computing (EUC), 2017 IEEE International Conference on*, Vol. 1. IEEE, 183–190.
- [59] Haifeng Yu and Amin Vahdat. 2000. Design and Evaluation of a Continuous Consistency Model for Replicated Services. In *4th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*. Article 21, 14 pages.
- [60] Steve Zdancewic and Andrew C. Myers. 2003. Observational Determinism for Concurrent Program Security. In *16th IEEE Computer Security Foundations Workshop (CSFW)*. 29–43.
- [61] Steve Zdancewic, Lantian Zheng, Nathaniel Nystrom, and Andrew C. Myers. 2002. Secure Program Partitioning. *ACM Trans. on Computer Systems* 20, 3 (Aug. 2002), 283–328.